

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY**Seventh Semester of B. Tech. (IT) Examination****November 2022****IT443 Language Processors****Date: 21.11.2022, Monday****Time: 01:30 p.m. To 4:30 p.m.****Maximum Marks: 70****Instructions:**

1. The question paper comprises of two sections.
2. Section I and II must be attempted in separate answer sheets.
3. Make suitable assumptions and draw neat figures wherever required.

SECTION – I**Q-1 Do as directed.**

- 1) Generate regular expression for the given language of all strings $\Sigma \in \{a, b\}$, that begin and end with aa or bb. **[01]**
- 2) List out the components of Finite Automata. **[01]**
- 3) Which phase of the compiler is performing DFA minimization? **[01]**
- 4) Which technique combines the multiple loops into the single loop during code optimization phase? **[01]**
- 5) Define: Semantic gap **[01]**
- 6) Write a nested macro for calculating average of three integer numbers. **[02]**

Q-2 (a) Describe Language Processing activities in detail. **[04]****(b) Attempt Following Questions (Any Two). **[10]****

- 1) Explain two phases of assembler in detail.
- 2) Demonstrate constant folding and constant propagation techniques with suitable example.
- 3) Construct NFA for the following regular expression. Convert it to DFA using Subset Construction method.

$$(0+1)^* (00+11)$$

Q-3 (a) Construct NFA- $\wedge$ for following regular expression and convert it into DFA using Thompson's Construction. **[04]**

$$(0^* / 1^*)^*$$

(b) Attempt Following Questions (Any Two).

[10]

- 1) Why lexical analyzer is separated than syntax analyzer phase? Illustrate the role of lexical analyzer in compilation process with diagram.
- 2) Show the importance of symbol table in compiler design. How is the symbol table manipulated at various phases of compilation?
- 3) Define three address code. Which phase of the compiler generates three address code? Design three address code for given code.

$a = 2 * m + n$
while a do
 $a = a - n$

SECTION – II

Q - 4 Do as directed.

- 1) Define the following term with respect to bottom-up parsing technique: “Handle”. **[01]**
- 2) Select the appropriate option: **[01]**
LR stands for _____.
(a) Left to right
(b) Right to left scanning
(c) Left to right scanning
(d) Left to right scanning and right most derivation in reverse
- 3) List out different assembly language statements. **[01]**
- 4) Compare and contrast shift-reduce conflict and reduce-reduce conflict in brief. **[02]**
- 5) Classify Operator Grammar with suitable example. **[02]**

Q-5 (a) Answer following questions.

[06]

- 1) Construct the parse tree and identify the following grammar is ambiguous or not.

G: { $S \rightarrow 0Y \mid 01$
 $X \rightarrow 0XY \mid 0$
 $Y \rightarrow XY1 \mid 1$ }

- 2) Analyze the following grammar and identify the type of the grammar.

G: { $S \rightarrow aS \mid bA$
 $A \rightarrow aA \mid a$ }

(b) Answer following questions (Any Two)

[08]

- 1) Calculate the first and follow functions for the given context-free grammar

G: { $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$ }

- 2) Consider the following grammar and remove the Left recursion to generate non-left recursive grammar.

$$G: \{ S \rightarrow Aa \mid b \\ A \rightarrow Sc \mid d \}$$

- 3) Consider the following context-free grammar where the set of terminals are $\{a,b,c,d,f\}$. Generate the LL(1) parsing table.

$$G: \{ S \rightarrow daT \mid Rf \\ T \rightarrow aS \mid baT \mid \epsilon \\ R \rightarrow caTR \mid \epsilon \}$$

- Q-6 (a)** List down the bottom-up parsing techniques. Distinguish between simple-LR, Canonical-LR and Lookahead-LR techniques in detail. **[06]**

- (b) Answer following questions (Any Two)** **[08]**

- 1) For the given context-free grammar,

$$E \rightarrow E + E \mid E * E \mid id$$

Construct the operator precedence parser and parse the string $id + id * id$.

- 2) Prepare the Left factoring of following context-free grammar.

$$A \rightarrow ad \mid a \mid ab \mid abc \mid b$$

- 3) Construct LR(0) item-set of SLR bottom-up parsing technique for following grammar.

$$E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid id$$
